

# HealthPro · Звіт про рефакторинг

Етап 4 завершено: 4-В аналітика, 4-Г історія+експорт, 4-Д налаштування

Дата: 24 квітня 2026 р.  
Проект: HealtPro-Moie-Zdorovia (PWA, Vite + Capacitor)

Етап 4 повністю завершено

## Короткий підсумок

Усю логіку, що залишалася в `src/app.js`, розкладено по feature-модулях. Файл `app.js` перетворено на тонкий оркестратор: реєстрація обробників подій, маршрутизація сторінок, делегування дій. Жодних регресій у UI, консоль чиста.

| Метрика                             | До   | Після | Зміна |
|-------------------------------------|------|-------|-------|
| Рядків у <code>src/app.js</code>    | 1652 | 254   | −1398 |
| Feature-модулів                     | 3    | 8     | +5    |
| Файлів у <code>src/features/</code> | ~7   | 24    | +17   |
| Самостійних файлів модулів          | 14   | 34    | +20   |

Це означає, що всі домени застосунку (тиск, графіки, ліки, кроки, аналітика, історія, експорт, налаштування, PWA) тепер живуть окремо одне від одного й готові до підключення нативних плагінів Capacitor.

## Етап 4-В · features/analytics

Винесено всю аналітику дашборду: інтегральний показник здоров'я, ІМТ, рекомендації та модальне вікно тренду.

### Створені файли

- `analytics/health-score.js` — `calcHealthScore` (зважений скор тиску, пульсу, ліків, ІМТ, активності), `getDetailedScores`, `toggleHealthTooltip`.
- `analytics/bmi.js` — `calcBMI`, `getBMICategory`, `renderBMI` з кольоровим бейджем.
- `analytics/recommendations.js` — `RECO_SVG`, `generateAdvice` (10+ правил для різних рівнів тиску, пульсу, ваги), `renderRecommendations`, `toggleReco`.
- `analytics/trend-modal.js` — `openTrendModal`, `closeTrendModal` з SVG-графіком тренду.
- `analytics/index.js` — `renderAnalytics`: збирає всі картки дашборду (скор, середнє тиску, тренд, ВООЗ, всього вимірів, прихильність до прийому ліків, макс/мін, пульс, % норми) та делегує підпотрібним модулям.

### Інтеграція

- `features/steps/index.js` доповнено експортом `getStepCount()`, щоб модуль скору міг порахувати «активність» без імпорту `state` з кроків.
- `renderAnalytics` викликається на події `measurement:saved` та при перемиканні мови.

## Етап 4-Г · features/history та features/export

Окремо винесено журнал вимірів (з фільтром тиждень / місяць / усі), а також усі

способи поділитися даними: JSON, CSV, друкована веб-версія та повний PDF-звіт для лікаря.

## history/

- history/index.js — setHistoryPeriod, renderHistory, deleteMeasurement, clearHistory. Діалог підтвердження видалення.
- Видалення емітить подію measurement:deleted → app.js перерисовує графік і заголовок.

## export/

- export/csv.js — exportData (JSON-бекап), exportCSV (повний CSV всіх вимірів), exportReportCSV (CSV за обраний період), importData (відновлення зі збереженого JSON).
- export/modal.js — openExportModal, selectExportPeriod (тиждень / 2 тижні / місяць / довільний), updateExportCount.
- export/print.js — printReportPeriod: повноцінний друкований звіт (HTML-вікно з SVG-графіком, таблицею, інформацією про пацієнта, місцем для підпису лікаря).
- export/pdf.js — exportPDF: збирає високоякісний PDF через html2canvas + jsPDF із chart-snapshot, картками статистики, таблицями вимірів і ліків, юридичним дисклеймером.
- export/logo.js — base64-логотип винесено в окремий файл, щоб не засмічувати модулі.
- export/index.js — публічні експорти.

## Чому модуль export бере showPage як параметр

PDF-звіт мусить тимчасово переключити сторінку на «тиск», щоб зчитати canvas графіка. Замість прямого імпорту з app.js (циклічна залежність) ми зареєстрували showPage через setPDFShowPage(showPage) у точці входу. Модуль залишається ізольованим і легко тестується.

## Етап 4-Д · features/settings та features/pwa

Останній великий блок: профіль користувача, мовний шар, тема, нагадування про прийом ліків і вимір тиску, дисклеймер та все, що пов'язано з PWA-обгорткою.

## settings/

- settings/theme.js — applyTheme, toggleTheme з іконкою сонце / місяць.
- settings/i18n.js — setLang, t, renderDisclaimerBody, renderWelcomeText. Шина registerReRender(fn) дозволяє іншим модулям підписатися на зміну мови без циклічних імпортів.
- settings/profile.js — saveProfile, loadProfileFields, updateHeader (ПІБ, вік, зріст, вага, контакти, нормальні значення тиску і пульсу, екстрений контакт).
- settings/notifications.js — toggleNotifications (Push API + дозволи), toggleMeasureReminder, saveReminderTimes, scheduleNotifications (запобігає подвійному спрацюванню за допомогою \_firedReminders).
- settings/data.js — clearAllData з підтвердженням і повним скиданням до defaultSettings.
- settings/disclaimer.js — повна історія прийнять (LocalStorage), DISCLAIMER\_VERSION = «1.0», міграція зі старого ключа, openDisclaimerModal, acceptDisclaimer, checkDisclaimer.

## pwa/

- pwa/index.js — installApp, registerSW (з пропуском у режимі розробки), applyUpdate (для нової версії SW), setupOnlineIndicator (бейдж «офлайн»). Listener beforeinstallprompt реєструється на завантаженні модуля.

# Архітектура після рефакторингу

Усі модулі спілкуються через явні імпорти або через шину подій on/emit з core/state.js. Жодного звертання до глобальних змінних, жодних DOM-обробників напрямую у бізнес-логіці.

## Дерево src/

```
src/
├── app.js                (254 рядки — оркестратор)
├── core/
│   ├── state.js         (state, saveData, on/emit, setToast, today, DB)
│   ├── storage.js       (defaultSettings, ключі LocalStorage)
│   └── utils.js          (formatTime, formatDate, todayISO, avg)
├── i18n/
│   └── index.js          (T_UK, T_RU, WELCOME_T, DISCLAIMER_T, PDF_LABELS)
├── features/
│   ├── pressure/        (norm, who, critical, index)
│   ├── charts/          (helpers, bp-chart, index)
│   ├── meds/            (drug-db, index)
│   ├── steps/           (index)
│   ├── analytics/       (health-score, bmi, recommendations, trend-modal, index)
│   ├── history/         (index)
│   ├── export/          (csv, modal, pdf, print, logo, index)
│   ├── settings/        (theme, i18n, profile, notifications, data, disclaimer, index)
│   └── pwa/             (index)
```

## Зв'язки між модулями

- Усі модулі читають дані з core/state.js, мутують масиви через push/splice (не через переприсвоєння).
- showToast визначається у app.js (бо потребує DOM) і реєструється у state.js через setToast().
- Подія measurement:saved → перерисовка графіка та аналітики; measurement:deleted → перерисовка графіка та заголовка.
- registerReRender(fn) — кожен модуль, що залежить від мови, підписується один раз; setLang викликає всі обробники.
- setPDFShowPage(showPage) — модуль експорту отримує функцію навігації як inversion-of-control.

## Перевірка якості

- Vite dev сервер стартував чисто (Vite 5.4.21, порт 5000, ready in 696 ms).
- Браузерна консоль чиста: тільки інформаційні повідомлення Vite та сховища.
- Welcome-екран відображається коректно з усіма елементами (логотип, опис, перемикач мов UA/RU, посилання на дисклеймер, кнопка згоди).
- Жодного звернення до глобальних змінних app.js у інших файлах — підтверджено пошуком.

# Етап 5: рекомендація локальної бази даних

Зараз HealthPro зберігає всі дані у localStorage браузера. Це працює для PWA, але має критичні обмеження саме для нативного Android/iOS-білду через Capacitor.

## Чому localStorage не підходить для нативного застосунку

- Ліміт ~5-10 МБ, на iOS WebView ще менший. Багаторічна історія вимірів та фотографії не помістяться.
- Синхронні API блокують головний потік під час запису, що відчутно на старіших телефонах.
- Немає індексів і запитів — фільтри (тиск > 140 за останні 90 днів) робляться лінійним проходом масиву.
- Дані можна стерти, очистивши кеш WebView; для мед-додатка це неприпустимо.
- Немає атомарних транзакцій — у разі краху під час запису можна побити журнал.

## Рекомендація: @capacitor-community/sqlite

### Перший вибір

- Повноцінний SQLite на пристрої: до 2 ГБ, шифрування (AES-256), індекси, транзакції, повноцінні SQL-запити.
- Обгортка для Android (через android.database.sqlite) та iOS (через системний SQLite). Один однаковий API через Capacitor.
- Web-fallback на sql.js (чистий JS у IndexedDB) — той самий код працює і в PWA в браузері, і в нативі.
- Вбудована підтримка експорту/імпорту бази як файла .db — можна резервно копіювати на iCloud / Google Drive.
- Активна підтримка спільнотою, оновлюється для нових версій Capacitor 6/7.

## Запропонована схема (мінімум)

```
CREATE TABLE measurements (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  ts INTEGER NOT NULL,          -- мс UTC  
  sys INTEGER NOT NULL,  
  dia INTEGER NOT NULL,  
  pulse INTEGER,  
  note TEXT  
);  
CREATE INDEX idx_measurements_ts ON measurements(ts DESC);
```

```
CREATE TABLE pills (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  name TEXT NOT NULL,  
  dose TEXT,  
  time TEXT,                  -- HH:MM  
  days TEXT,                  -- daily | mon,tue | date  
  date INTEGER                -- ts якщо days = date  
);
```

```
CREATE TABLE pills_taken (  
  date TEXT NOT NULL,         -- YYYY-MM-DD  
  pill_id INTEGER NOT NULL,  
  taken INTEGER NOT NULL DEFAULT 0,  
  PRIMARY KEY (date, pill_id)  
);
```

```
CREATE TABLE settings (  

```

```
key TEXT PRIMARY KEY,  
value TEXT  
);
```

### Альтернативи (нижчий пріоритет)

- @capacitor/preferences — Key-Value (NSUserDefaults / SharedPreferences). Підходить тільки для дрібних налаштувань, не для журналу вимірів.
- @capacitor/filesystem + JSON-файли — простий, але без транзакцій і без запитів. Ризик пошкодити файл при краху під час запису.
- Dexie.js (IndexedDB) — чудовий у браузері, але на iOS WebView IndexedDB обнуляється під час очищення кешу. Недостатньо надійно для мед-додатка.
- Realm / WatermelonDB — потужні, але важкі для нашого розміру даних і додають велику залежність.

### План впровадження (Етап 5)

- 5-А · Додати @capacitor-community/sqlite, створити core/db/sqlite.js (open, migrate, runQuery, getRows).
- 5-Б · Написати міграцію localStorage → SQLite (одноразова, з прогресом і резервною копією).
- 5-В · Замінити state.measurements / state.pills / state.pillsTaken на запити з кешуванням у пам'яті (state стає read-only after-load).
- 5-Г · Оновити модулі pressure / meds / history / analytics, щоб писали через сервіс БД (зберігає сумісність API).
- 5-Д · Додати резервне копіювання у файл .db (експорт/імпорт) і опціональне шифрування.